

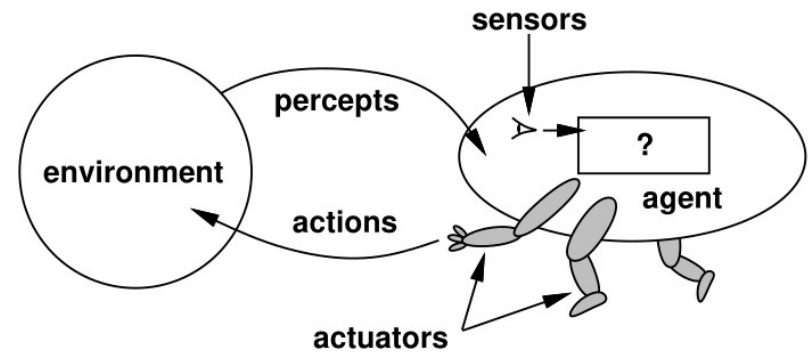
Intelligent Agents

Chapter-2

Outline

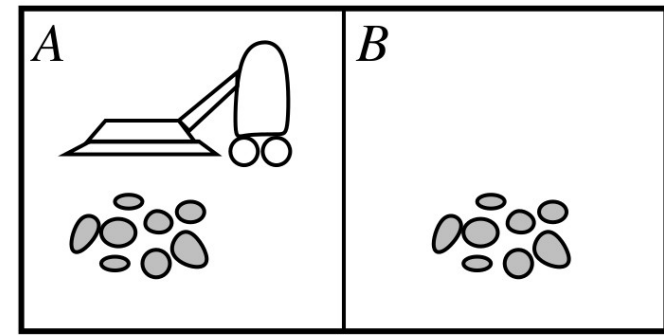
- Agents and Environments
- Rationality
- PEAS (Performance measure, Environment, Actuators, Sensors)
- Environment Types
- Agent Types

Agents and Environments



- An Agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.
- Agents include humans, robots, soft-bots, thermostats, etc
- Percept = Single input from environment
- Percept sequence = Complete sequence of every thing the agent received as input
- The agent function maps from percepts histories to actions
- $F : P^* \rightarrow A$
- The Agent program runs on the physical architecture to produce F

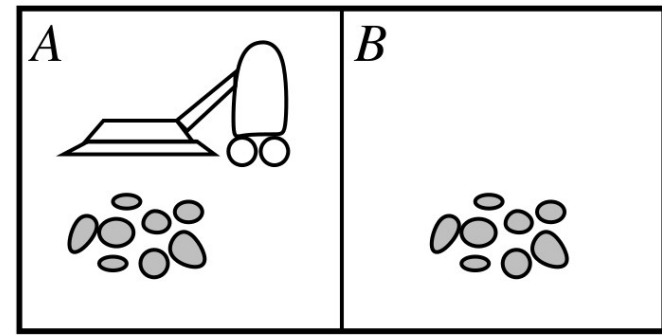
Vacuum Cleaner World



- Vacuum Cleaner World: To clean two tiles. If A is clean turn Right, if B is clean turn Left.
- Percepts : Location and contents e.g. [A, Dirty], [B, Clean]
- Actions: Left, Right, Suck, NoOp

Percept sequence	Action
<i>[A, Clean]</i>	<i>Right</i>
<i>[A, Dirty]</i>	<i>Suck</i>
<i>[B, Clean]</i>	<i>Left</i>
<i>[B, Dirty]</i>	<i>Suck</i>
<i>[A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>

Vacuum Cleaner World



- What is the correct function?
- Can it be implemented in a small agent program?

```
function REFLEX-VACUUM-AGENT( [location, status]) returns an action  
  if status = Dirty then return Suck  
  else if location = A then return Right  
  else if location = B then return Left
```

Rationality

- Fixed performance measure evaluates the environment sequence
 - – one point per square cleaned up in time T ?
 - – one point per clean square per time step, minus one per move?
 - – penalize for $> k$ dirty squares?
- A **rational agent** chooses whichever action maximizes the expected value of the performance measure given the percept sequence to date
- Rational $\neq$ omniscient (Knowing every thing)
 - – percepts may not supply all relevant information
- Rational $\neq$ clairvoyant (Having super natural ability)
 - – action outcomes may not be as expected
- Hence, rational $\neq$ successful
- Rational $\Rightarrow$ exploration, learning, autonomy

Rationality

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

- **Exploration**
 - Information gathering, Exploration can be done
- **Learning**
 - Learn as much as possible from what it perceives
- **Autonomy**
 - The extent that an agent relies on the prior knowledge of its designer rather than on its own percepts, we say that the agent lacks autonomy.

PEAS

- To design a rational agent, we must specify the task environment
- **Performance:**
- **Environment:**
- **Actuators:**
- **Sensors:**
- Consider e.g. the task of designing an automated taxi:

PEAS

- To design a rational agent, we must specify the task environment
- Consider e.g. the task of designing an automated taxi:
 - **Performance:** Safety, Destination, Profits, Legality, Comfort, ...
 - **Environment:** US Streets, Freeways, traffic, pedestrians, weather, ...
 - **Actuators:** Steering, Accelerator, Brake, Horn, Speaker, Display, ...
 - **Sensors:** Camera, Accelerometer, Gauges, Engine sensors, Keyboard, GPS, Lidar, Radar, etc

PEAS

- Consider e.g. the task of internet shopping agent
- **Performance:** Price, Quality, Appropriateness, Efficiency, ...
- **Environment:** Current and future www sites, vendors, shippers, ...
- **Actuators:** Display to users, follow URL, Fill forms ...
- **Sensors:** HTML page (text, graphics, scripts ...)

PEAS

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 2.5 Examples of agent types and their PEAS descriptions.

Properties of Environments

- **Observability:**
- **No of Agents:**
- **Competitive:**
- **Determinism:**
- **Episodic, Sequential:**
- **Static vs Dynamic:**
- **Discrete vs Continuous:**
- **Known vs Unknown:**

Properties of Environments

- **Observability:** Full observable, Partial observable, unobservable
 - Access to complete, partial or no state of the environment
 - Agents goals may still be achievable with un-observability
- **No of Agents:** Single agent, Multiple agents
 - An agent solving a crossword puzzle by itself is a single agent environment
 - An agent playing chess is a two-agent environment.
 - Which entities should be treated as an agent?
- **Competitive Environment:**
 - Chess is a competitive multi-agent environment
 - Taxi driver agent is a partially cooperative multiagent environment.

Properties of Environments

- **Determinism:** Deterministic vs Stochastic
 - If next state of environment depends on the current state and the actions of the agent, then we say that the environment is deterministic
 - Else it is stochastic.
 - In Vacuum cleaner example, randomly appearing dirt, or unreliable suction mechanism can be considered as stochastic elements.
- **Un Certain:**
 - Environment is uncertain if it is not fully observable or not deterministic.

Properties of Environments

- **Episodic vs Sequential:**

- **Episodic:** The agent's experience is divided into atomic episodes.
- Next episode does not depend on actions taken in previous episode.
- Defect detection in assembly line is episodic.
- **Sequential:** The current decisions could affect all future decisions.
- Chess and Taxi Driving are sequential i.e. short term actions can have long term consequences.

Properties of Environments

- **Static vs Dynamic:**

- **Dynamic** : If the environment can change while an agent is deliberating.
- **Static**: Environment not changing. Easy to handle.

- **Discrete vs Continuous:**

- Applies to states of environment, way time is handled, percepts and actions
- Chess has discrete set of environments, and percepts and actions.
- Taxi-driving has continuous state and time problem

- **Known vs Unknown**

- Agents (or designer's) knowledge about environment.
- Different from observability.

Properties of Environments

- **Hardest cases are**
 - Partially observable
 - Multi-agent
 - Stochastic
 - Sequential
 - Dynamic
 - Continuous
 - Unknown
- **Taxi Driving has all the features except that its a known problem.**

Properties of Environments

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

Agent Types

- **Four basic types in order of increasing generality:**
 - Simple reflex agents
 - Reflex agents with state
 - Goal based agents
 - Utility based agents
- All the above can be turned into learning agents

Agent Types – Simple Reflex Agents

- **Simple Reflex agents**

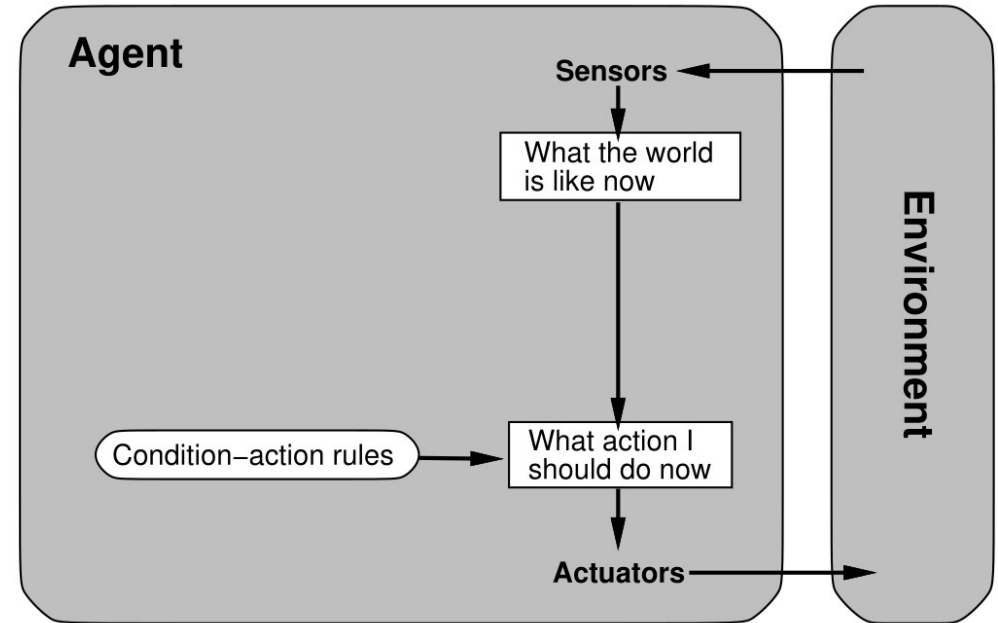
- That selects actions on the basis of the current percept, ignoring rest of the percept history.

- **Examples**

- Vacuum cleaner, as it takes decision on the basis of current location, and current dirt.

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
```

```
if status = Dirty then return Suck  
else if location = A then return Right  
else if location = B then return Left
```



Agent Types – Simple Reflex Agents

- **Simple Reflex behavior occurs in more complex environment also**
 - If car in front brakes, its red light appears → then you should notice and initiate your brake
 - This is called as **condition-action-rule**
 - **If** *car in front is braking*, **then** *initiate braking*.
 - Humans also have lots of condition-action-rules
- Hc

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition-action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Agent Types – Simple Reflex Agents

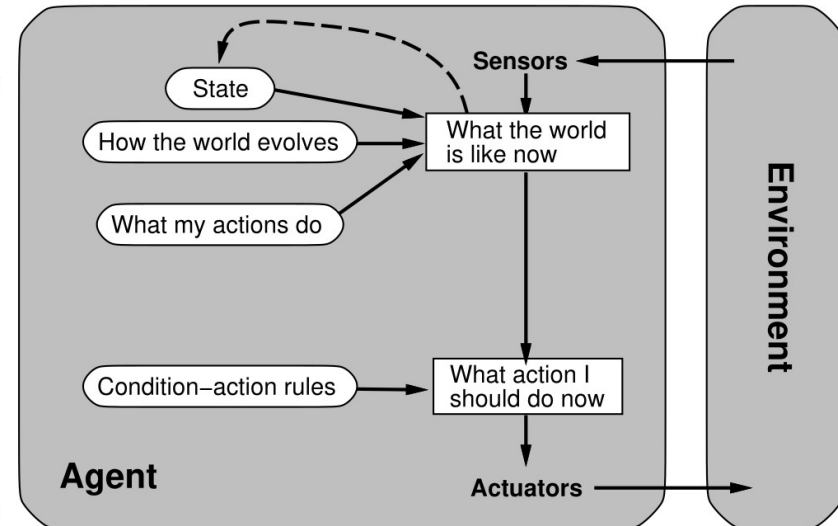
- Simple Reflex agents have an admirable property of being simple.
- But have limited intelligence
 - Correct decision to be made if the rule matches i.e. fully observable environment
- Example:
 - Difficult to determine from a single frame if front car is braking.
 - May fall into infinite loop in partially observable environments
- Randomized reflex agent mostly outperforms simple deterministic reflex agent.

Agent Types – Model/State based Reflex Agents

- Agent to keep track of the partial observability through internal state.
- Internal state keeps track of the past history
- Model is the information about the world.
- Update state function varies and depends upon the environment
-

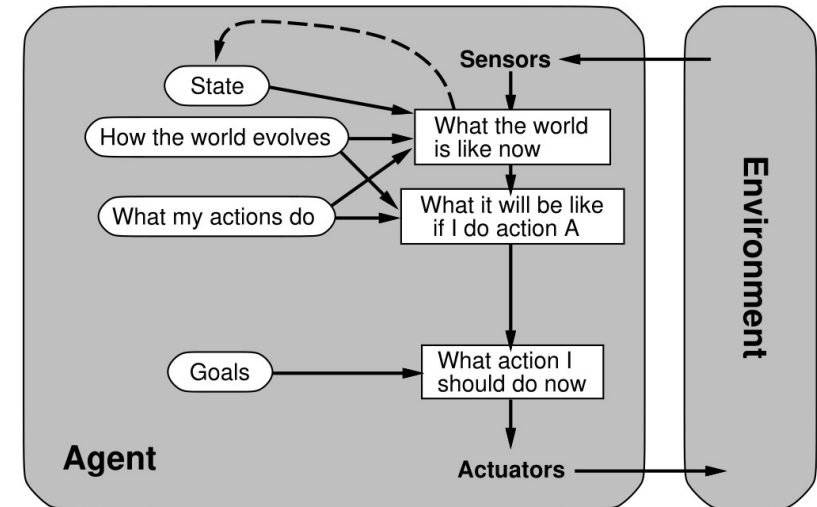
```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               model, a description of how the next state depends on current state and action
               rules, a set of condition–action rules
               action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```



Agent Types – Goal based Agents

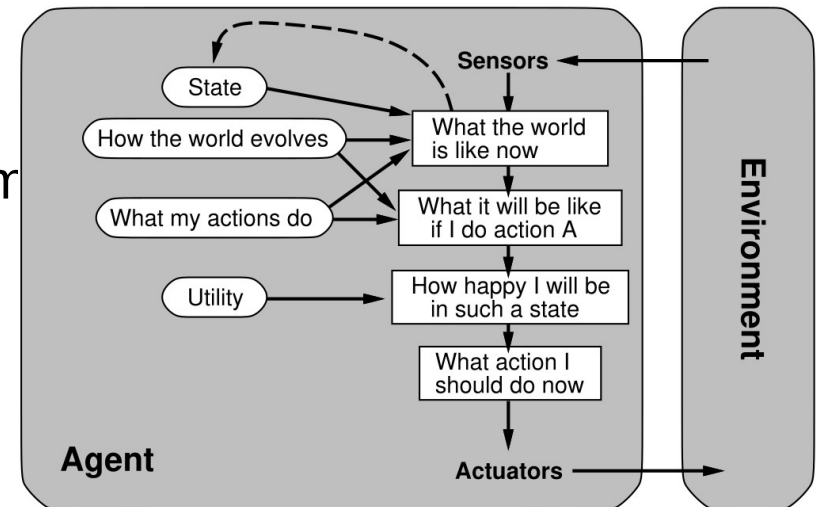
- The agent needs goal information that describe the desired situation
- No condition-action rules.
- Search and Planning are the sub-fields of the AI



Agent Types – Utility based Agents

- Only goal information is not the most rational choice.
- We need the best way to reach the goal.
- Utility function is an internationalization of performance measure.
- Much better performance of utility agents when
 - There are conflicting goals i.e. only few goals can be achieved.
 - There are several goals, and none of them can be achieved with certainty.

Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.



Agent Types – Learning Agents

- Learning agents divided into four conceptual components
 - **Learning Element:** Responsible for making improvements.
 - **Performance Element:** Selects external actions.
 - **Critic:** How the model is performing. Feedback loop to performance element
 - **Problem Generator:** Suggests actions that will lead to new learning.

